

PMSCS 670

Software Testing

Types of Testing

Lecture 4

Mohammad Ashraful Islam

Testing & Debugging

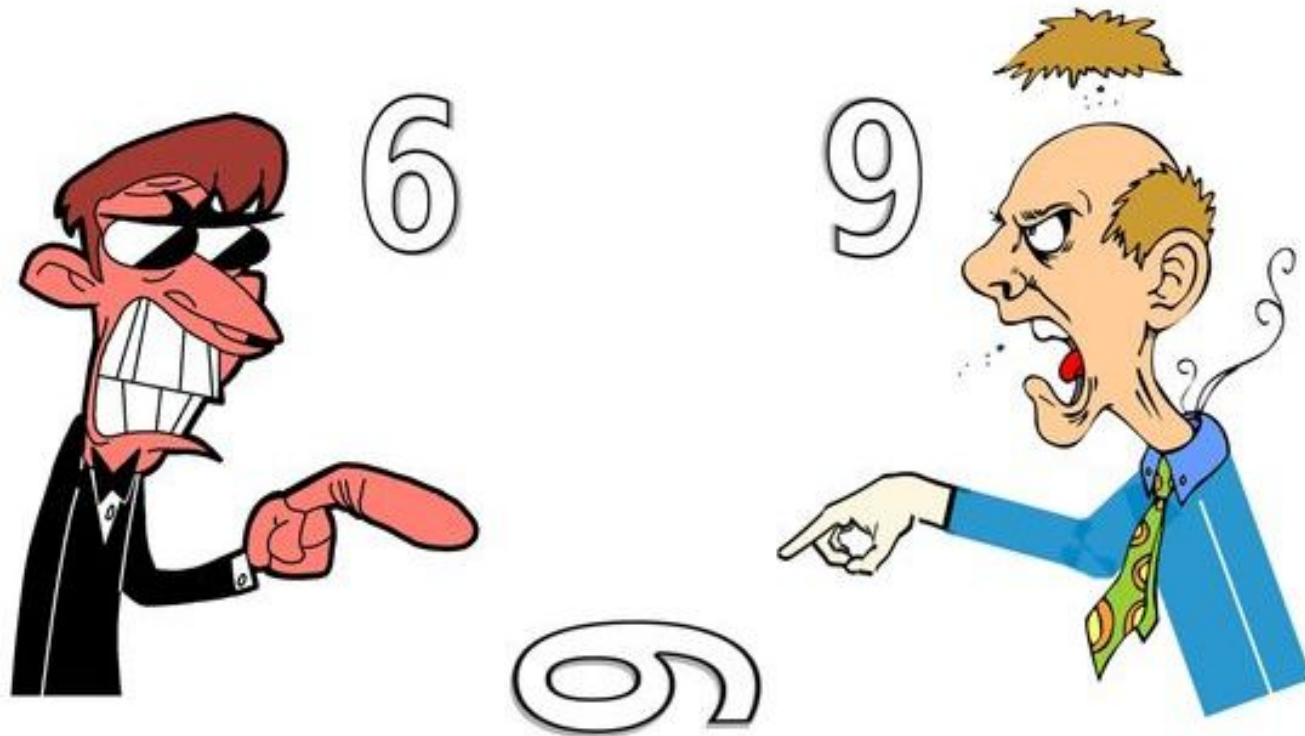
- **Testing** : Evaluating software by observing its execution
- **Test Failure** : Execution of a test that results in a software failure
- **Debugging** : The process of finding a fault given a failure

Not all inputs will “trigger” a fault into causing a failure

Testing vs Debugging

Testing	Debugging
1. Testing always starts with known conditions, uses predefined methods, and has predictable outcomes too.	1. Debugging starts from possibly un-known initial conditions and its end cannot be predicted, apart from statistically.
2. Testing can and should definitely be planned, designed, and scheduled.	2. The procedures for, and period of, debugging cannot be so constrained.
3. It proves a programmers failure.	3. It is the programmer's vindication.
4. It is a demonstration of error or apparent correctness.	4. It is always treated as a deductive process.
5. Testing as executed should strive to be predictable, dull, constrained, rigid, and inhuman.	5. Debugging demands intuitive leaps, conjectures, experimentation, and some freedom also.
6. Much of the testing can be done without design knowledge.	6. Debugging is impossible without detailed design knowledge.
7. It can often be done by an outsider.	7. It must be done by an insider.
8. Much of test execution and design can be automated.	8. Automated debugging is still a dream for programmers.
9. Testing purpose is to find failure .	9. Debugging purpose is to find the root cause of the failure .
10. Some popular testing tools: JUnit , Selenium , Jmeter , KatalonStudio , TestComplete etc	10. Some popular Debugger tools: WinDbg , OllyDbg , IDA Pro , etc

Developer vs Tester



Principles of Software Testing

- There are **seven principles** of Software Testing.
 1. Testing shows presence of defects
 2. Exhaustive testing is impossible
 3. Early testing reduces the cost
 4. Defect clustering
 5. Pesticide paradox
 6. Testing is context dependent
 7. Absence of error – fallacy

Principles of S/W Testing Cont...

- **Testing Shows Presence of Defects:** Testing shows the presence of defects in the software. The **goal of testing is to make the software fail**. Sufficient testing reduces the presence of defects. In case **testers are unable to find defects** after repeated regression testing **doesn't mean that the software is bug-free**.
- **Exhaustive Testing is Impossible:** Testing all the functionalities using all valid and invalid inputs and preconditions is known as Exhaustive testing. If we keep on testing **all possible test conditions** then the software **execution time and costs will rise**. So instead of doing exhaustive testing, **risks and priorities will be taken into consideration** whilst doing testing and estimating testing efforts.

Principles of S/W Testing Cont...

- **Early Testing:** Defects detected in early phases of SDLC are less expensive to fix. So conducting early testing reduces the cost of fixing defects. It **is cheaper to change the incorrect requirement** compared to **fixing the fully developed functionality** which is not working as intended.
- **Defect Clustering:** Defect Clustering in software testing means that a **small module or functionality contains most of the bugs** or it has the most operational failures. As per the Pareto Principle (80-20 Rule), 80% of issues comes from 20% of modules and remaining 20% of issues from remaining 80% of modules. So we do emphasize testing on the 20% of modules where we face 80% of bugs.

Principles of S/W Testing Cont...

- **Pesticide Paradox:** The process of repeating the same test cases again and again, eventually, the same test cases will no longer find new bugs is called Pesticide Paradox in software testing. So to overcome this Pesticide Paradox, it is necessary to review the test cases regularly and add or update them to find more defects.
- **Testing is Context Dependent:** Testing approach depends on the context of the software we develop. We do test the software differently in different contexts. For example, online banking application requires a different approach of testing compared to an e-commerce site.

Principles of S/W Testing Cont...

- **Absence of Error – Fallacy:** The **absence of error does not necessarily mean that the software is error free.** 99% of bug-free software may still be unusable, if wrong requirements were incorporated into the software and the software is not addressing the business needs. The software which we built not only be a 99% bug-free software but also it must fulfill the business needs otherwise it will become an unusable software. The **testing team must start with the hypothesis that there are errors in the software.** Using test cases and other methods, the testing team desires to prove the hypothesis wrong.

Software Assessment

- Two types-
 - **Static Analysis**: based on factual system documents available such as system requirement study document, design document, and source code.
 - **Dynamic Analysis**: based on the behavioral and performance properties.

Types of Software Testing

- *Given below is the list of some common types of Software Testing:*

Functional Testing include:

- Unit Testing
- Integration Testing
- System Testing
- Sanity Testing
- Smoke Testing
- Interface Testing
- Regression Testing
- Acceptance Testing
- Localization Testing

Non-functional Testing include:

- Performance Testing
- Load Testing
- Stress Testing
- Volume Testing
- Security Testing
- Compatibility Testing
- Install Testing
- Recovery Testing
- Reliability Testing
- Usability Testing
- Compliance Testing

Functional vs Non Functional Test.

- Functional testing verifies each function/feature of the software whereas Non Functional testing verifies non-functional aspects like performance, usability, reliability, etc.
- Functional testing can be done manually whereas Non Functional testing is hard to perform manually.
- Functional testing is based on customer's requirements whereas Non Functional testing is based on customer's expectations.
- Functional testing has a goal to validate software actions whereas Non Functional testing has a goal to validate the performance of the software.

Functional vs Non Functional Test.

- A Functional Testing example is to check the login functionality whereas a Non Functional testing example is to check the dashboard should load in 2 seconds.
- Functional describes what the product does whereas Non Functional describes how the product works.
- Functional testing is performed before the non-functional testing.

Alpha Testing

- It is the most common type of testing used in the Software industry. The objective of this testing is to **identify all possible issues or defects before releasing** it into the market or to the user.
- Alpha Testing is **carried out at the end of the software development phase** but before the Beta Testing. Still, minor design changes may be made as a result of such testing.
- Alpha Testing is **conducted at the developer's site**. In-house virtual user environment can be created for this type of testing.

Acceptance Testing

- An Acceptance Test is performed by the client **and verifies whether the end to end flow of the system is as per the business requirements or not** and if it is as per the needs of the end-user. Client accepts the software only when all the features and functionalities work as expected.
- It is the last phase of the testing, after which the software goes into production. This is also called User Acceptance Testing (UAT).

Ad-hoc Testing

- The name itself suggests that this testing is performed on an Ad-hoc basis i.e. **with no reference to the test case and also without any plan or documentation** in place for such type of testing.
- The objective of this testing is to find the defects and break the application by executing any flow of the application or any random functionality.
- Ad-hoc Testing is an **informal way of finding defects and can be performed by anyone** in the project.
- It is difficult to identify defects without a test case but sometimes it is possible that defects found during ad-hoc testing might not have been identified using existing test cases.

Accessibility Testing

- The aim of Accessibility Testing is to **determine whether the software or application is accessible for disabled people or not.**
- Here, disability means deaf, color blind, mentally disabled, blind, old age and other disabled groups.
- Various checks are performed such as font size for visually disabled, color and contrast for color blindness, etc.

Beta Testing

- Beta Testing is a formal type of Software Testing which is **carried out by the customer**. It is performed in the Real Environment before releasing the product to the market for the actual end-users.
- Beta Testing is carried out **to ensure that there are no major failures in the software or product and it satisfies the business requirements from an end-user perspective**. Usually, this testing is typically done by end-users or others. The Beta version of the software or product is released to a limited certain number of users in a specific area.
- So end-user actually uses the software and shares the feedback to the company. Company then takes necessary action before releasing the software to the worldwide.

Back-end Testing

- Whenever an input or data is entered on front-end application, it stores in the database and the testing of such database is known as Database/Back-end Testing.
- There are different databases like SQL Server, MySQL, and Oracle, etc. Database Testing involves testing of table structure, schema, stored procedure and so on.
- In **Back-end Testing GUI is not involved**, testers are directly connected to the database with proper access and testers can easily verify data by running a few queries on the database.
- There can be issues identified like data loss, deadlock, data corruption etc during this back-end testing and these issues are critical to fixing before the system goes live into the production environment

Compatibility Testing

- It is a testing type in which it validates how software behaves and runs in a different environment, web servers, hardware, and network environment.
- Compatibility testing ensures that software can run on a different configuration, different database, different browsers, and their versions.
- Compatibility testing is performed by the testing team.

Browser Compatibility Testing

- It is a subtype of Compatibility Testing and is performed by the testing team.
- Browser Compatibility Testing is performed for web applications and it ensures that the software can run with the combination of different browser and operating system.
- This type of testing also validates whether web application runs on all versions of all browsers or not.

Backward Compatibility Testing

- It is a type of testing which validates whether the newly developed software or updated software works well with the older version of the environment or not.
- Backward Compatibility Testing checks whether the new version of the software works properly with file format created by an older version of the software; it also works well with data tables, data files, data structure created by the older version of that software.
- If any of the software is updated then it should work well on top of the previous version of that software.

Boundary Value Testing

- This type of testing checks the behavior of the application at the boundary level.
- Boundary Value Testing is performed for checking if defects exist at boundary values. Boundary Value Testing is used for testing a different range of numbers.
- There is an upper and lower boundary for each range and testing is performed on these boundary values.
- If testing requires a test range of numbers from 1 to 500 then Boundary Value Testing is performed on values at 0, 1, 2, 499, 500 and 501.

Equivalence Partitioning

- It is a testing technique and a type of Black Box Testing. During this Equivalence Partitioning, a set of the group is selected and a few values or numbers are picked up for testing. It is understood that all values from that group generate the same output.
- The aim of this testing is to remove redundant test cases within a specific group which generates the same output but not any defect.
- Suppose, the application accepts values between -10 to +10 so using equivalence partitioning the values picked up for testing are zero, one positive value, one negative value. So the Equivalence Partitioning for this testing is -10 to -1, 0, and 1 to 10.

Graphical User Interface (GUI) Testing

- The objective of this GUI Testing is **to validate the GUI as per the business requirement**. The expected GUI of the application is mentioned in the Detailed Design Document and GUI mockup screens.
- The GUI Testing includes the size of the buttons and input field present on the screen, alignment of all text, tables, and content in the tables.
- It also validates the menu of the application, after selecting different menu and menu items, it validates that the page does not fluctuate and the alignment remains same after hovering the mouse on the menu or sub-menu.

Gorilla Testing

- Gorilla Testing is a testing type performed by a tester and sometimes by the developer the as well.
- In Gorilla Testing, one module or the functionality in the module is tested thoroughly and heavily.
- The objective of this testing is to check the robustness of the application.

Happy Path Testing

- The objective of Happy Path Testing is to test an application successfully on a positive flow.
- It does not look for negative or error conditions.
- The focus is only on the valid and positive inputs through which application generates the expected output.

Incremental Integration Testing

- Incremental Integration Testing is a Bottom-up approach for testing i.e continuous testing of an application when new functionality is added.
- Application functionality and modules should be independent enough to test separately. This is done by programmers or by testers.

Install/Uninstall Testing

- Installation and Uninstallation Testing is done on full, partial, or upgrade install/uninstall processes on different operating systems under different hardware or software environment.

Load Testing

- It is a type of Non-Functional Testing and the objective of Load Testing is to check how much load or maximum workload a system can handle without any performance degradation.
- Load Testing helps to find the maximum capacity of the system under specific load and any issues that cause software performance degradation.
- Load testing is performed using tools like JMeter, LoadRunner, WebLoad, Silk performer, etc.

Monkey Testing

- Monkey Testing is carried out by a tester assuming that if the monkey uses the application then how random input, values will be entered by the Monkey without any knowledge or understanding of the application.
- The objective of Monkey Testing is to check if an application or system gets crashed by providing random input values/data.
- Monkey Testing is performed randomly and no test cases are scripted and it is not necessary to be aware of the full functionality of the system.

Mutation Testing

- Mutation Testing is a type of white box testing in which the **source code of one of the program is changed and verifies whether the existing test cases can identify these defects in the system.**
- The change in the program source code is very minimal so that it does not impact the entire application, only the specific area having the impact and the related test cases should be able to identify those errors in the system.

Negative Testing

- Testers having the mindset of “attitude to break” and using Negative Testing they validate that if system or application breaks.
- A Negative Testing technique is performed using incorrect data, invalid data or input.
- It validates that if the system throws an error of invalid input and behaves as expected.

Recovery Testing

- It is a type of testing which validates **how well the application or system recovers from crashes or disasters**.
- Recovery Testing determines if the system is able to continue the operation after a disaster.
- Assume that application is receiving data through the network cable and suddenly that network cable has been unplugged. Sometime later, plug the network cable; then the system should start receiving data from where it lost the connection due to network cable unplugged.

Security Testing

- It is a type of testing performed by a special team of testers. A system can be penetrated by any hacking way.
- Security Testing is done to check how the software or application or website is secure from internal and external threats.
- This testing includes how much software is secure from the malicious program, viruses and how secure and strong the authorization and authentication processes are.
- It also checks how software behaves for any hackers attack and malicious programs and how software is maintained for data security after such a hacker attack.

Stress Testing

- This testing is done when a system is stressed beyond its specifications in order to check how and when it fails.
- This is performed under heavy load like putting large number beyond storage capacity, complex database queries, continuous input to the system or database load.

Usability Testing

- Under Usability Testing, User-friendliness check is done.
- The application flow is tested to know if a new user can understand the application easily or not. Proper help documented if a user gets stuck at any point.
- Basically, **system navigation is checked in this testing.**

Volume Testing

- Volume Testing is a type of Non-Functional Testing performed by the Performance Testing team.
- The software or application undergoes a huge amount of data and **Volume Testing checks the system behavior and response time of the application when the system came across such a high volume of data.**
- This high volume of data may impact the system's performance and speed of the processing time.

Smoke Testing

- Whenever a new build is provided by the development team then the Software Testing team validates the build and ensures that no major issue exists.
- The testing team ensures that the build is stable and a detailed level of testing is carried out further. Smoke Testing checks that no show stopper defect exists in the build which will prevent the testing team to test the application in detail.
- If testers find that the major critical functionality is broken down at the initial stage itself then testing team can reject the build and inform accordingly to the development team.
- Smoke Testing is carried out to a detailed level of any Functional or Regression Testing.

Sanity Testing

- Sanity Testing is done to determine if a new software version is performing well enough to accept it for a major testing effort or not.
- If an application is crashing for the initial use then the system is not stable enough for further testing. Hence a build or an application is assigned to fix it.

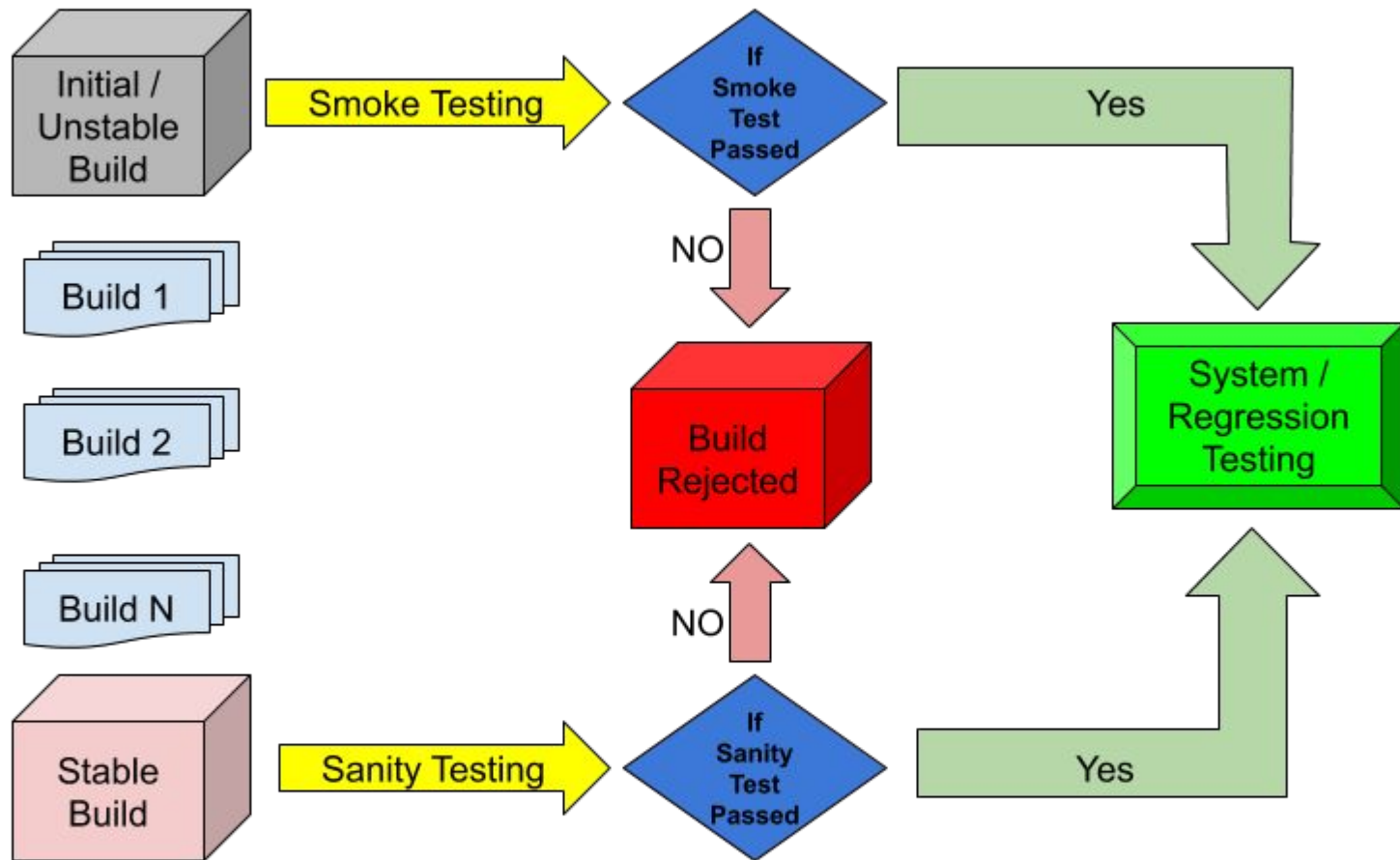
Regression Testing

- Testing an application as a whole for the modification in any module or functionality is termed as Regression Testing.
- It is difficult to cover all the system in Regression Testing, so typically Automation Testing Tools are used for these types of testing.

Smoke vs Sanity vs Regression

Smoke Testing	Sanity Testing	Regression Testing
Performed on initial builds	Performed on stable builds	Performed on stable builds
Tests the stability of a new build	Tests the stability of a new functionality or code changes in the existing build	Tests the functionality of all affected areas after new functionality /code changes in the existing build
Covers end-to-end basic functionalities	Covers certain modules , in which code changes have been made	Covers detailed testing targeting all the affected areas after new functionalities are added
Executed by testers & sometimes also by developers	Executed by testers	Executed by testers, mostly via automation
A part of basic testing	A part of regression testing	Regression Testing is a superset of Smoke and Sanity Testing

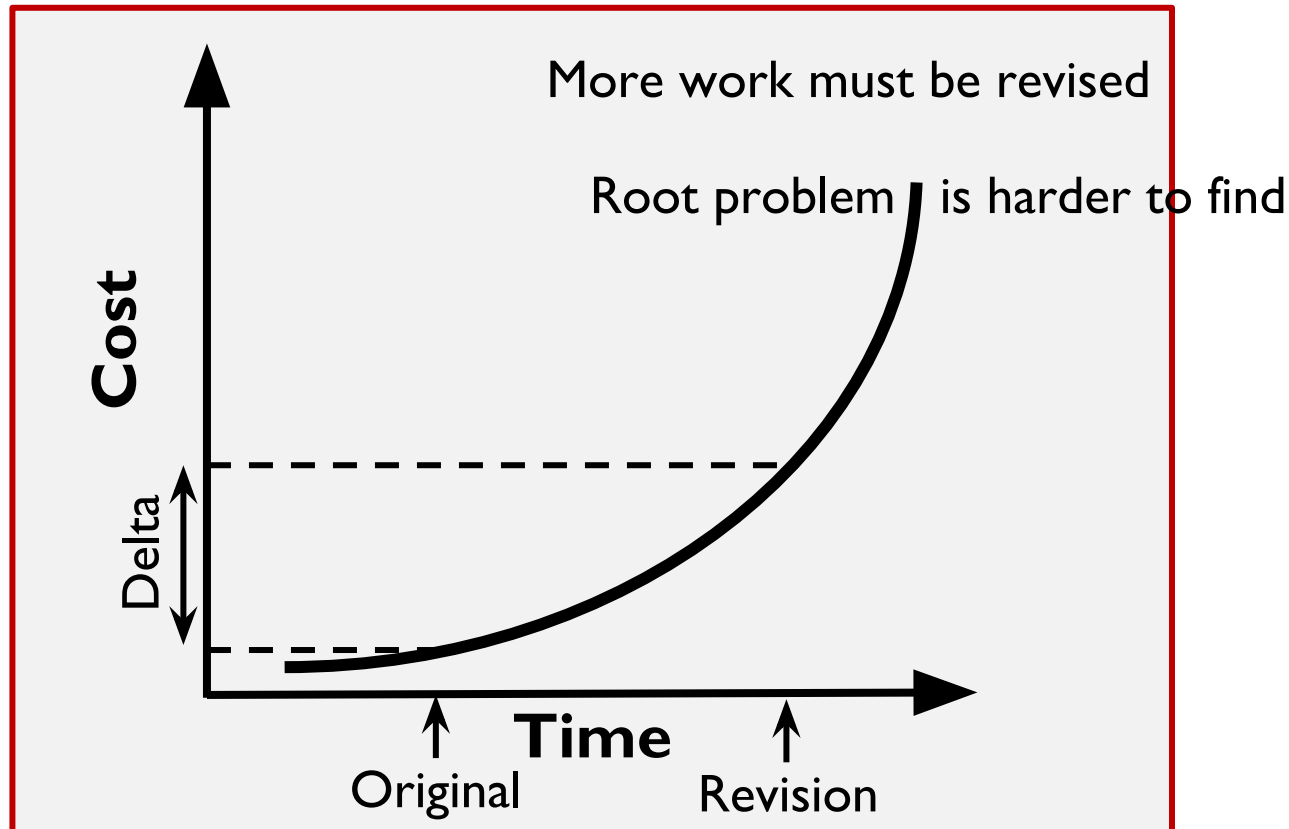
Smoke vs Sanity vs Regression



Copyright © Testsigma Technologies

The Increased Emphasis on Testing

- Philosophy of traditional software development methods
 - Upfront analysis
 - Extensive modeling
 - Reveal problems as early as possible



Traditional Assumptions

1. Modeling and analysis can identify potential problems early in development
 2. Savings implied by the cost-of-change curve justify the cost of modeling and analysis over the life of the project
- These are true **if requirements are always complete and current**
 - But those annoying customers keep changing their minds!
 - Humans are naturally good at **approximating**
 - But pretty bad at **perfecting**
 - These two assumptions have made software engineering frustrating and difficult for decades

Thus, agile methods ...

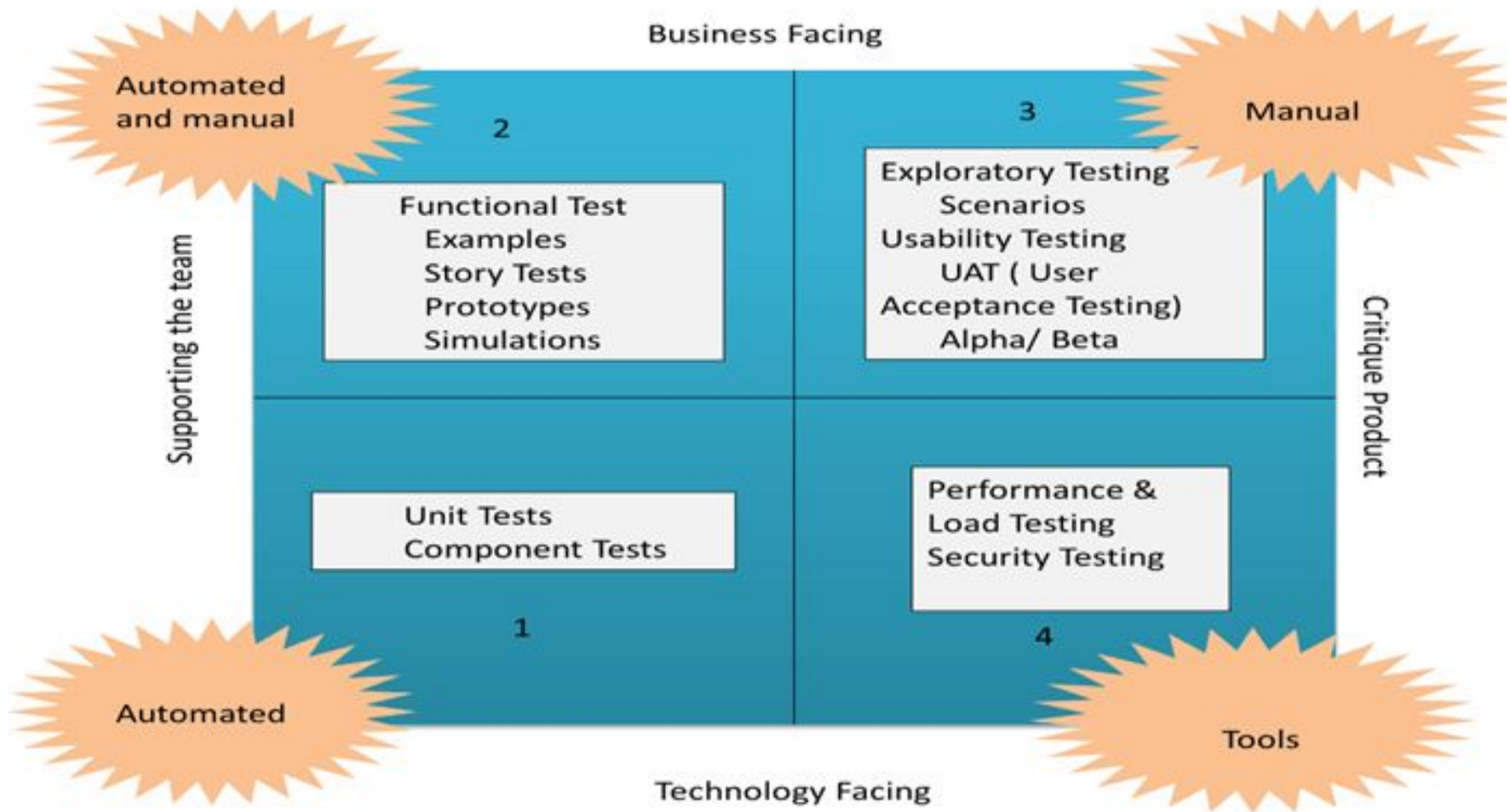
Why Be Agile ?

- Agile methods start by recognizing that **neither assumption** is valid for many current software projects
 - Software engineers are **not good at developing requirements**
 - We do not anticipate many **changes**
 - Many of the changes we do anticipate are **not needed**
- Requirements (and other “non-executable artifacts”) tend to go **out of date** very quickly
 - We seldom take time to **update** them
 - Many current software projects **change continuously**
- Agile methods expect software to **start small and evolve** over time
 - Embraces **software evolution** instead of fighting it

Manifesto for Agile

- The *Manifesto for Agile Software Development* is based on twelve principles
 1. Customer satisfaction by early and continuous delivery of valuable software.
 2. Welcome changing requirements, even in late development.
 3. Deliver working software frequently (weeks rather than months)
 4. Close, daily cooperation between business people and developers
 5. Projects are built around motivated individuals, who should be trusted
 6. Face-to-face conversation is the best form of communication (co-location)
 7. Working software is the primary measure of progress
 8. Sustainable development, able to maintain a constant pace
 9. Continuous attention to technical excellence and good design
 10. Simplicity—the art of maximizing the amount of work not done—is essential
 11. Best architectures, requirements, and designs emerge from self-organizing teams
 12. Regularly, the team reflects on how to become more effective, and adjusts accordingly

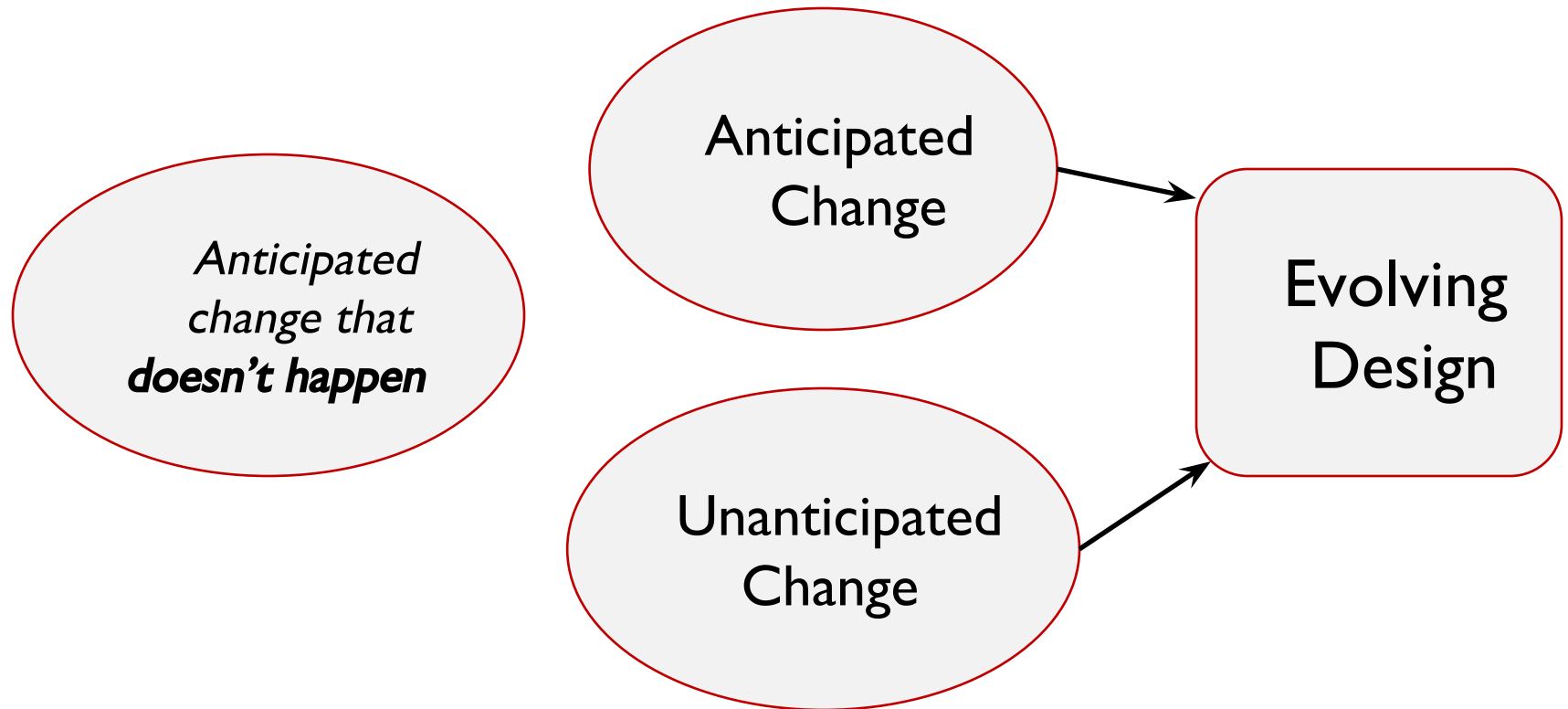
Manifesto for Agile



Supporting Evolutionary Design

Traditional design advice says to anticipate changes

Designers often anticipate changes that don't happen



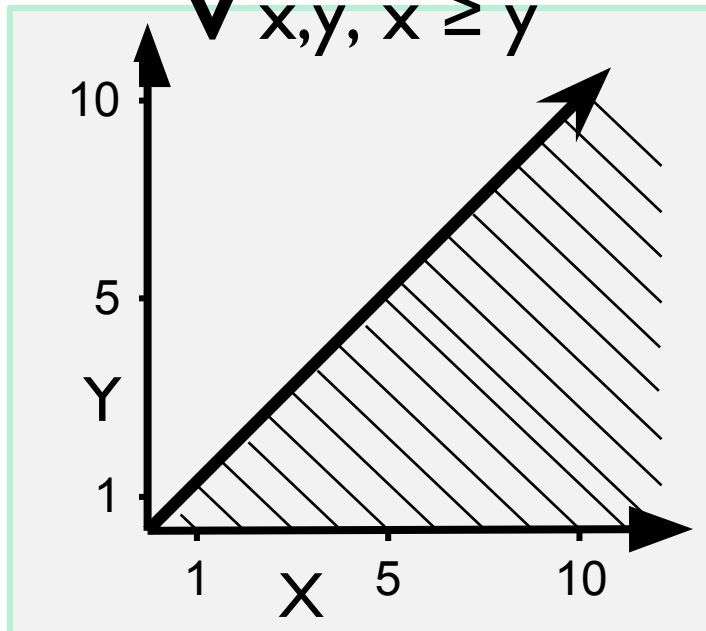
Both **anticipated and unanticipated** changes affect design

The Test Harness as Guardian (4.2)

What is Correctness ?

Traditional
Correctness
(Universal)

$$\forall x, y, x \geq y$$



Agile Correctness
(Existential)

{ (1, 1) ☐ T
(1, 0) ☐ T
(0, 1) ☐ F
(10, 5) ☐ T
(10, 12) ☐ F }

Comparative View of Correctness

- In **traditional** methods, we try to define **all correct behavior** completely, at the beginning
 - What is **correctness**?
 - Does “correctness” **mean anything** in large engineering products?
 - People are **VERY BAD** at completely defining correctness
- In **agile** methods, we redefine correctness to be **relative** to a specific set of tests
 - If the software behaves correctly **on the tests**, it is “correct”
 - Instead of defining all behaviors, we **demonstrate some** behaviors
 - **Mathematicians** may be disappointed at the lack of completeness

But software engineers ain't mathematicians!

Test Harnesses Verify Correctness

A *test harness* runs all automated tests efficiently and reports results to the developers

- Tests must be **automated**
 - Test automation is a **prerequisite** to test driven development
- Every test must include a **test oracle** that can evaluate whether that test executed correctly
- The tests replace the **requirements**
- Tests must be **high quality** and must run quickly
- We run tests **every time** we make a change to the software

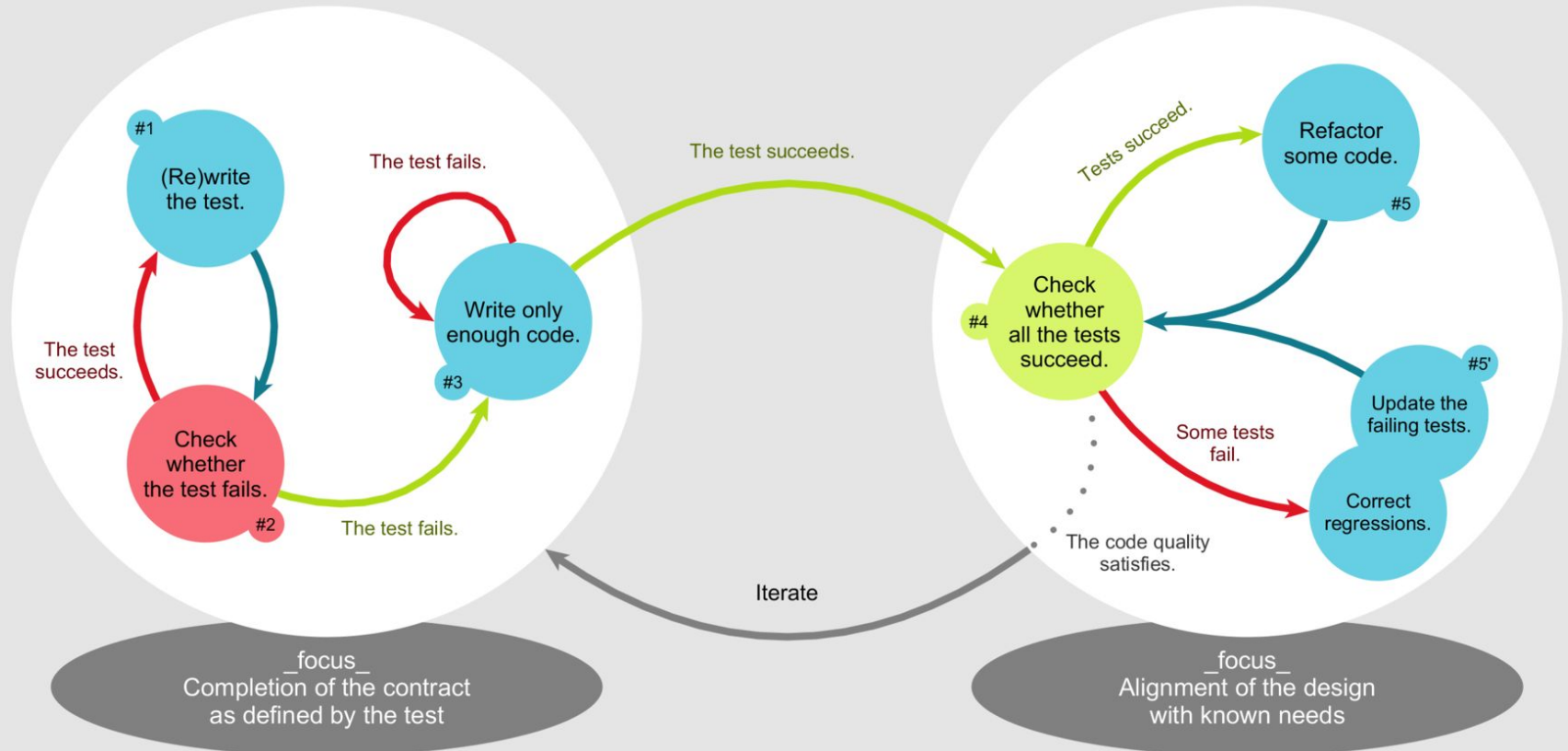
Test Driven Development (TDD)

- Steps of TDD:
 - Add a Test
 - Run all tests and see if the new test fails.
 - Write the code
 - Run Tests
 - Refactor Code
 - Repeat

Test Driven Development (TDD)

CODE-DRIVEN TESTING

REFACTORING



TEST-DRIVEN DEVELOPMENT



Xavier Pigeon

Continuous Integration

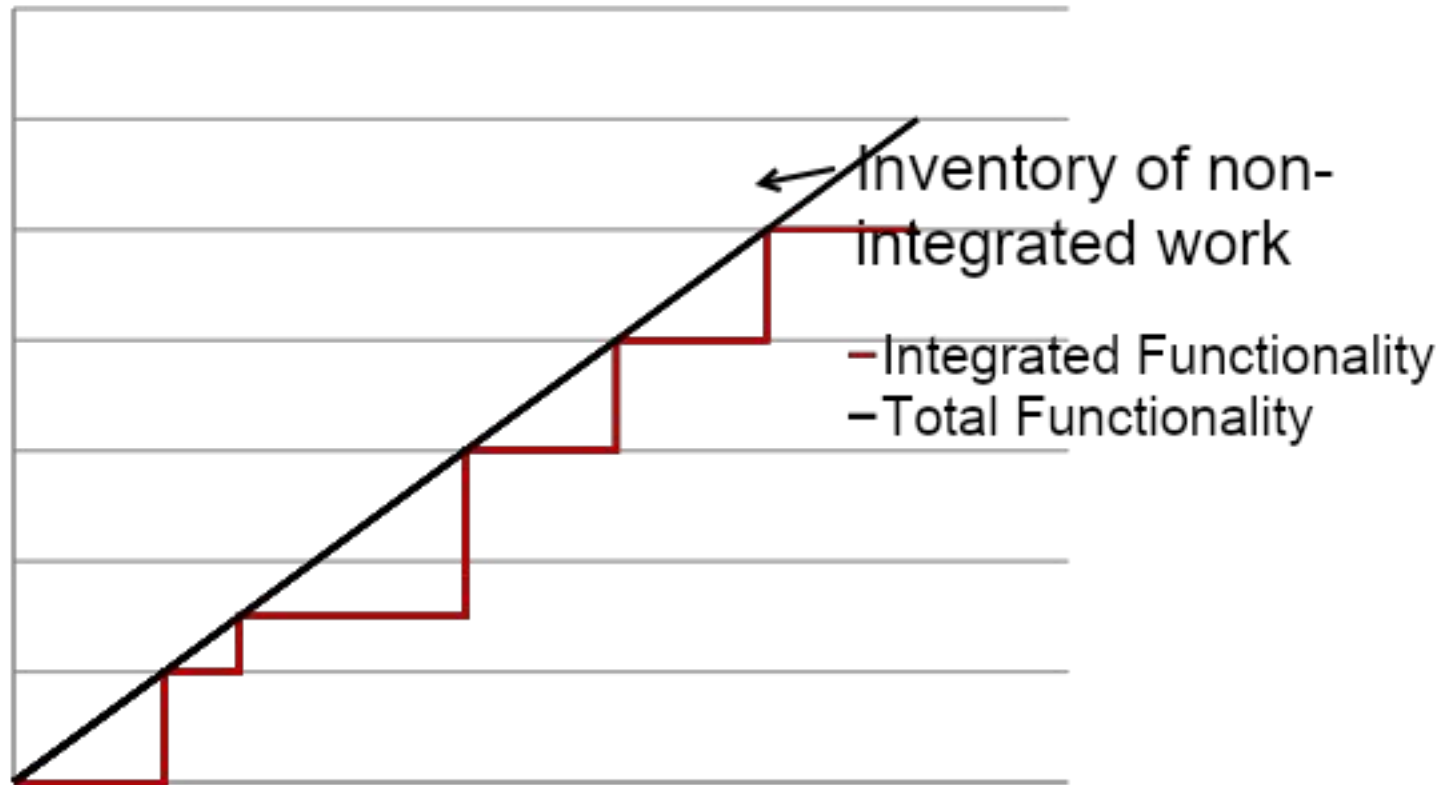
- Agile methods work best when the **current version** of the software can be **run against all tests at any time**

A **continuous integration server** rebuilds the system, returns, and re-verifies tests whenever *any* update is checked into the repository

- Mistakes are caught earlier
- Other developers are aware of changes early
- The rebuild and reverify must happen as soon as possible
 - Thus, tests need to execute quickly

A **continuous integration server** doesn't just run tests, it **decides if a modified system is still correct**

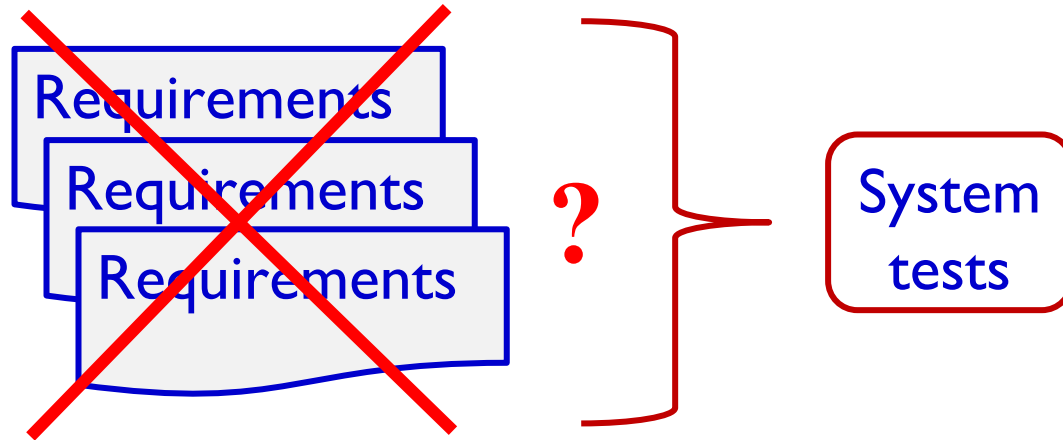
Continuous Integration Reduces Risk



Non-integrated functionality is dangerous!

System Tests in Agile Methods

Traditional testers often design system tests from requirements



But ... what if there are **no** traditional requirements documents ?

User Stories

A *user story* is a few sentences that captures what a user will do with the software

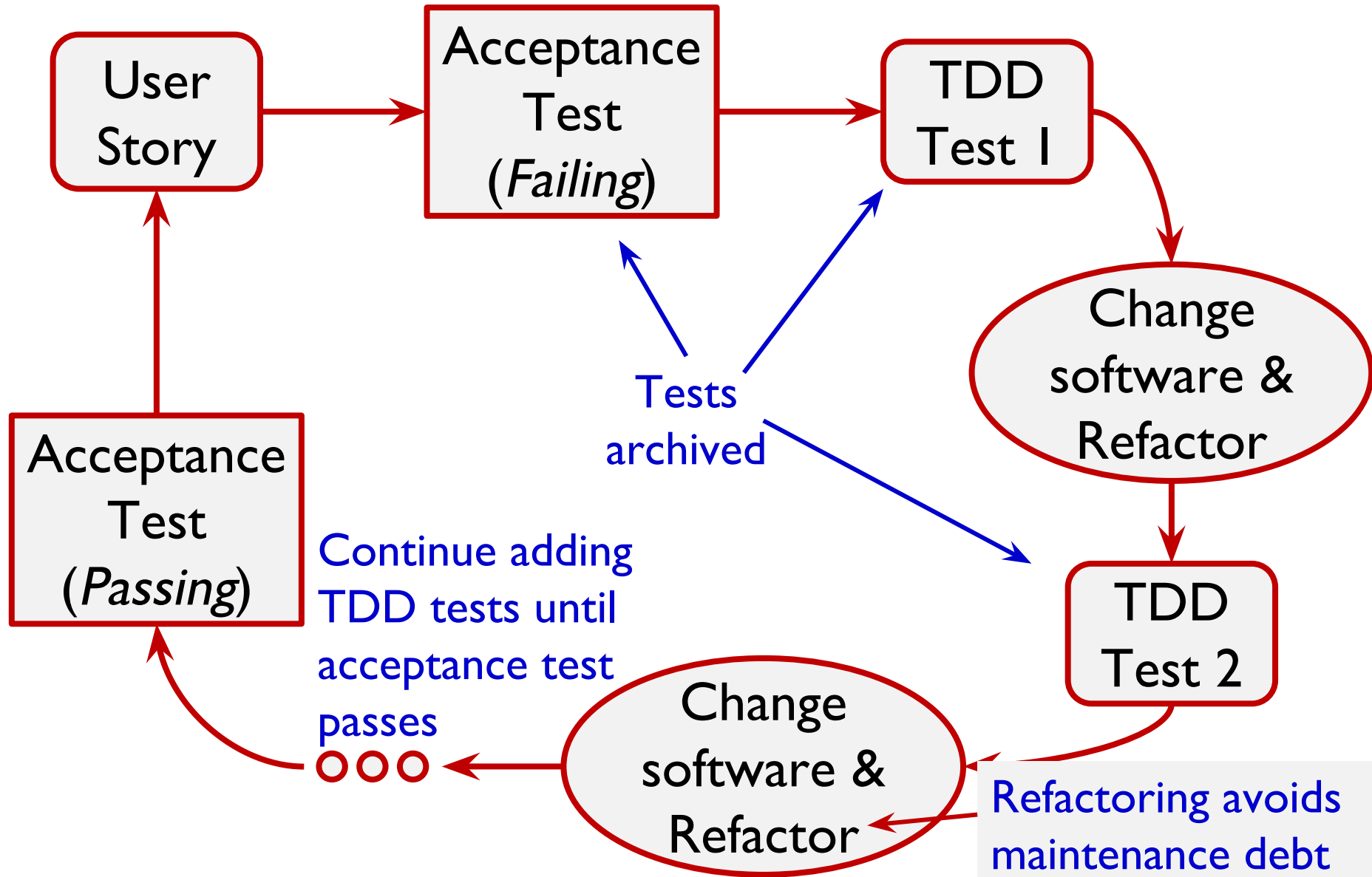
Withdraw money from
checking account

Agent sees a list of today's
interview applicants

Support technician sees
customer's history on
demand

- In the language of the **end user**
- Usually small in scale with **few details**
- Not archived

Acceptance Tests in Agile Methods



Adding Tests to Existing Systems

- Most of today's software is **legacy**
 - No legacy **tests**
 - Legacy requirements hopelessly **outdated**
 - Designs, if they were ever written down, **lost**
- Companies sometimes **choose not to change** software out of fear of failure

How to apply TDD to legacy software with no tests?

- Create an entire new test set? — too expensive!
- Give up? — a mixed project is unmanageable

Incremental TDD

- When a change is made, add TDD tests for just that change
 - Refactor
- As the project proceeds, the collection of TDD tests continues to grow
- Eventually the software will have strong TDD tests

Challenges with agile Development

- a) Chances of error are more in agile, as documentation is given less priority, eventually puts more pressure on QA team
- b) New features are introduced quickly, which reduces the available time for test teams to identify whether the latest features are according to the requirement and does it truly address the business suits
- c) Testers are often required to play a semi-developer roled
- d) Test execution cycles are highly compressed
- e) Very less time to prepare test plan
- f) For regression testing, they will have minimal timing
- g) Change in their role from being a gate-keeper of quality to being a partner in Quality
- h) Requirement changes and updates are inherent in an agile method, becoming the biggest challenge for QA

The Testing Shortfall

- Do **TDD tests** (acceptance or otherwise) test the software well?
 - Do the tests achieve good **coverage** on the code?
 - Do the tests find most of the **faults**?
 - If the software passes, should management feel confident the software is **reliable**?

NO!



Why Not?

- Most agile tests focus on “*happy paths*”
 - What should happen under normal use
- They often miss things like
 - Confused-user paths
 - Creative-user paths
 - Malicious-user paths

The agile methods literature
does not give much guidance

Design Good Tests

1. Use a human-based approach

- Create additional user stories that describe non-happy paths
- How do you know when you're finished?
- Some people are very good at this, some are bad, and it's hard to teach



Next
Lectures

2. Use modeling and criteria

- Model the input domain to design tests
- Model software behavior with graphs, logic, or grammars
- A built-in sense of completion
 - Much easier to teach—engineering
 - Requires discrete math knowledge

Summary

- More companies are putting **testing first**
- This can dramatically **decrease cost** and **increase quality**
- A different view of “***correctness***”
 - Restricted but practical
- Embraces **evolutionary design**
- TDD is definitely **not** test automation
 - Test automation is a **prerequisite** to TDD
- **Agile tests** aren't enough